

ANALYSIS OF NON-RECURSIVE ALGORITHMS

Contents

1	Introduction	3
2	Steps for Analysing a Non-Recursive Algorithm	3
3	Problem 1: Finding Minimum / Maximum Value	4
3.1	1. Basic Input	5
3.2	2. Basic Operation	5
3.3	3. Number of Executions	5
3.4	4. Set Up Summation Equation	5
3.5	5. Solve and Find Big-O	6
4	Problem 2: Symmetric Check in a Matrix	6
4.1	1. Basic Input	6
4.2	2. Basic Operation	7
4.3	3. Number of Executions	7
4.4	4. Set Up Summation Equation	7
4.5	5. Solve the Summation	7
5	Problem 3: Linear Search	8
5.1	1. Basic Input	8
5.2	2. Basic Operation	8
5.3	3. Number of Executions	8
5.4	4. Set Up Summation Equation	9
5.5	5. Solve and Find Big-O	9
6	Problem 4: Unique Elements	9
6.1	1. Basic Input	10
6.2	2. Basic Operation	10
6.3	3. Number of Executions	10
6.4	4. Set Up Summation Equation	10
6.5	5. Solve the Summation	11
7	Problem 5: Matrix Multiplication	11
7.1	1. Basic Input	12
7.2	2. Basic Operation	12
7.3	3. Number of Executions	12
7.4	4. Set Up Summation Equation	13
7.5	5. Solve and Find Big-O	13
8	Comparison of All Problems	13

9	Important Summation Formulas	13
9.1	Constant Summation	14
9.2	Sum of First n Natural Numbers	14
9.3	Sum of Squares	14
10	General Analysis Template	14

1 Introduction

A **non-recursive algorithm** is an algorithm that does not call itself.

For example:

```
for(int i = 0; i < n; i++)
{
    // statements
}
```

To analyse the time complexity of a non-recursive algorithm, we usually identify the **basic operation** and count how many times it is executed.

2 Steps for Analysing a Non-Recursive Algorithm

We follow the following five main steps.

Step 1: Identify the Basic Input

Determine the size of the input.

For example:

$$A[0 \dots n - 1]$$

has:

$$n \text{ elements}$$

For an $n \times n$ matrix:

$$n^2 \text{ elements}$$

Step 2: Identify the Basic Operation

The **basic operation** is the operation that is executed most often or represents the main work of the algorithm.

Examples:

- Comparison
- Addition
- Multiplication
- Assignment

For example:

```
if(A[i] < min)
```

The basic operation is:

$$A[i] < min$$

Step 3: Count the Number of Executions

Determine how many times the basic operation is performed.

For example:

The comparison is performed approximately n times.

Step 4: Set Up the Summation

If the number of executions depends on the loop variables, write a summation.

For example:

$$\sum_{i=0}^{n-1} 1$$

Step 5: Solve the Summation

Using:

$$\sum_{i=0}^{n-1} 1 = n$$

we get:

$$T(n) = n$$

Therefore:

$$\boxed{O(n)}$$

3 Problem 1: Finding Minimum / Maximum Value

Consider the following algorithm for finding the minimum value.

```
Algorithm FindMin(A[0...n-1])

min = A[0]

for i = 1 to n-1 do
    if A[i] < min then
        min = A[i]

return min
```

The same analysis applies to finding the maximum value.

3.1 1. Basic Input

The input is an array:

$$A[0 \dots n - 1]$$

Therefore, the input size is:

$$\boxed{n}$$

elements.

3.2 2. Basic Operation

The important operation is the comparison:

$$\boxed{A[i] < min}$$

This comparison determines whether the current element is smaller than the current minimum.

For maximum:

$$A[i] > max$$

would be the basic operation.

3.3 3. Number of Executions

The loop starts from:

$$i = 1$$

and ends at:

$$i = n - 1$$

Therefore, the comparison is executed:

$$(n - 1) - 1 + 1$$

times.

Hence:

$$\boxed{n - 1}$$

comparisons are performed.

3.4 4. Set Up Summation Equation

We can also express this using summation:

$$T(n) = \sum_{i=1}^{n-1} 1$$

Using:

$$\sum_{i=l}^u 1 = u - l + 1$$

we get:

$$T(n) = (n - 1) - 1 + 1$$

$$T(n) = n - 1$$

3.5 5. Solve and Find Big-O

Therefore:

$$T(n) = n - 1$$

Ignoring the constant -1 :

$$T(n) = O(n)$$

Therefore, finding the minimum or maximum element takes:

$$O(n)$$

time.

4 Problem 2: Symmetric Check in a Matrix

Consider an $n \times n$ matrix.

A matrix is symmetric if:

$$A[i][j] = A[j][i]$$

for all appropriate i and j .

Algorithm:

```
Algorithm Symmetric(A[0...n-1][0...n-1])
for i = 0 to n-2 do
  for j = i+1 to n-1 do
    if A[i][j] != A[j][i] then
      return false
return true
```

4.1 1. Basic Input

The input is an:

$$n \times n$$

matrix.

Therefore, the number of elements is:

$$n^2$$

4.2 2. Basic Operation

The basic operation is the comparison:

$$\boxed{A[i][j] \neq A[j][i]}$$

4.3 3. Number of Executions

The outer loop is:

$$i = 0 \text{ to } n - 2$$

For a fixed value of i , the inner loop is:

$$j = i + 1 \text{ to } n - 1$$

Therefore, the number of executions of the inner loop for a fixed i is:

$$(n - 1) - (i + 1) + 1$$

Simplifying:

$$n - i - 1$$

Therefore, the total number of comparisons is:

$$n - i - 1$$

4.4 4. Set Up Summation Equation

The total number of comparisons is:

$$T(n) = \sum_{i=0}^{n-2} \sum_{j=1}^{n-1} (u - l + 1)$$

4.5 5. Solve the Summation

We know:

$$T(n) = \sum_{i=0}^{n-2} \sum_{j=1}^{n-1} (u - l + 1)$$

$$T(n) = \sum_{i=0}^{n-2} (n - 1 - (1) + 1)$$

$$T(n) = \sum_{i=0}^{n-2} (n - 1)$$

$$T(n) = n - 1(n - 2 - 0 + 1)$$

Therefore:

$$T(n) = n - 1(n - 1)$$

$$T(n) = n^2 - n$$

$$T(n) = O(n^2)$$

Therefore:

$$\text{Symmetric Matrix Check} = O(n^2)$$

5 Problem 3: Linear Search

Consider the linear search algorithm.

```

Algorithm LinearSearch(A[0...n-1], key)

for i = 0 to n-1 do
    if A[i] == key then
        return i

return -1

```

5.1 1. Basic Input

The input is an array:

$$A[0 \dots n - 1]$$

Therefore:

$$n \text{ elements}$$

and a search key.

5.2 2. Basic Operation

The basic operation is the comparison:

$$A[i] == key$$

5.3 3. Number of Executions

In the worst case, the key is:

- not present, or
- present at the last position.

Therefore, the comparison is performed:

$$n$$

times.

5.4 4. Set Up Summation Equation

The loop runs from:

$$i = 0$$

to:

$$i = n - 1$$

Therefore:

$$T(n) = \sum_{i=0}^{n-1} 1$$

Using:

$$\sum_{i=l}^u 1 = u - l + 1$$

we get:

$$T(n) = (n - 1) - 0 + 1$$

$$T(n) = n$$

5.5 5. Solve and Find Big-O

Therefore:

$$T(n) = n$$

Hence:

$$\text{Linear Search} = O(n)$$

6 Problem 4: Unique Elements

The following algorithm determines whether all elements in an array are distinct.

```
Algorithm UniqueElements(A[0...n-1])
```

```
for i = 0 to n-2 do
    for j = i+1 to n-1 do
        if A[i] == A[j] then
            return false
return true
```

6.1 1. Basic Input

The input is:

$$A[0 \dots n-1]$$

Therefore:

$$\boxed{n \text{ elements}}$$

6.2 2. Basic Operation

The basic operation is:

$$\boxed{A[i] == A[j]}$$

This comparison checks whether two elements are equal.

6.3 3. Number of Executions

The outer loop is:

$$i = 0 \text{ to } n-2$$

For every fixed i , the inner loop is:

$$j = i+1 \text{ to } n-1$$

Therefore, for a fixed i , the number of comparisons is:

$$(n-1) - (i+1) + 1$$

$$= n - i - 1$$

Therefore, the total number of comparisons is:

$$\boxed{\sum_{i=0}^{n-2} (n - i - 1)}$$

6.4 4. Set Up Summation Equation

The total number of comparisons is:

$$T(n) = \sum_{i=0}^{n-2} \sum_{j=1}^{n-1} (u - l + 1)$$

6.5 5. Solve the Summation

We know:

$$T(n) = \sum_{i=0}^{n-2} \sum_{j=1}^{n-1} (u - l + 1)$$

$$T(n) = \sum_{i=0}^{n-2} (n - 1 - (1) + 1)$$

$$T(n) = \sum_{i=0}^{n-2} (n - 1)$$

$$T(n) = n - 1(n - 2 - 0 + 1)$$

Therefore:

$$T(n) = n - 1(n - 1)$$

$$T(n) = n^2 - n$$

Therefore:

$$T(n) = O(n^2)$$

Hence:

$$\text{UniqueElements} = O(n^2)$$

This is the worst-case complexity, when all elements are distinct.

7 Problem 5: Matrix Multiplication

Consider multiplication of two $n \times n$ matrices:

$$C = A \times B$$

The standard matrix multiplication algorithm is:

```

Algorithm MatrixMultiplication(A, B, C)

for i = 0 to n-1 do
    for j = 0 to n-1 do
        C[i][j] = 0

        for k = 0 to n-1 do
            C[i][j] = C[i][j] + A[i][k] * B[k][j]

return C

```

7.1 1. Basic Input

The input consists of two matrices:

$$A[n \times n]$$

and

$$B[n \times n]$$

The output matrix is:

$$C[n \times n]$$

7.2 2. Basic Operation

The innermost loop performs:

$$C[i][j] = C[i][j] + A[i][k] \times B[k][j]$$

The basic operation can be considered the multiplication:

$$\boxed{A[i][k] \times B[k][j]}$$

or the complete multiply-add operation.

For counting the number of executions, both give the same order.

7.3 3. Number of Executions

There are three nested loops.

The first loop runs:

$$n$$

times.

The second loop runs:

$$n$$

times for every value of i .

The third loop runs:

$$n$$

times for every pair (i, j) .

Therefore, the total number of executions is:

$$n \times n \times n$$

$$\boxed{n^3}$$

7.4 4. Set Up Summation Equation

We can express the total number of basic operations using summation:

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$

Evaluate the innermost summation:

$$\sum_{k=0}^{n-1} 1 = n$$

Therefore:

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n$$

The middle summation gives:

$$T(n) = \sum_{i=0}^{n-1} n^2$$

Finally:

$$T(n) = n^3$$

7.5 5. Solve and Find Big-O

Therefore:

$$T(n) = n^3$$

Hence:

Matrix Multiplication = $O(n^3)$

8 Comparison of All Problems

Problem	Basic Operation	Time Complexity
Finding Min/Max	Comparison	$O(n)$
Symmetric Matrix	Matrix element comparison	$O(n^2)$
Linear Search	Key comparison	$O(n)$
Unique Elements	Pair comparison	$O(n^2)$
Matrix Multiplication	Multiplication / multiply-add	$O(n^3)$

9 Important Summation Formulas

The following formulas are useful while analysing non-recursive algorithms.

9.1 Constant Summation

$$\sum_{i=l}^u 1 = u - l + 1$$

Example:

$$\sum_{i=0}^{n-1} 1 = n$$

9.2 Sum of First n Natural Numbers

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Therefore:

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

Hence:

$$O(n^2)$$

9.3 Sum of Squares

Sometimes three nested loops result in:

$$\sum_{i=1}^n i^2$$

The formula is:

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

Therefore:

$$O(n^3)$$

10 General Analysis Template

For any non-recursive algorithm, use the following template:

1. Basic Input

What is the size of the input?

$$\text{Input size} = n$$

2. Basic Operation

Identify the operation that is performed repeatedly.

Examples:

$$A[i] < min$$

$$A[i] == key$$

$$A[i] == A[j]$$

$$A[i][k] \times B[k][j]$$

3. Number of Executions

Count how many times the basic operation is performed.

4. Set Up Summation

Write:

$$T(n) = \sum (\text{number of executions})$$

5. Solve the Summation

Use the appropriate formula.

Finally, write:

$$T(n) = O(\text{result})$$

**** © 2026 Pradeep Shet. All Rights Reserved.****